

ShopFlow Curriculum · Part A — Backend Engineering (World-Class Edition)

Part A — Backend Engineering

World-Class Edition — Production Architecture & Engineering Practice. Six integrated layers — OOP fundamentals, the Core Java toolkit, a production-grade Spring Boot & API platform, database and performance engineering, and cloud-native operability — built around two parallel tracks that run start to finish: a weekly Data Structures & Algorithms track and an Engineering Practice track (ADRs, code review, pairing, design docs) that turns every phase project into something a real engineering org would ship and defend.

5 phases, with 2 continuous cross-cutting tracks · 44 topics · 22-26 weeks · Beginner → Advanced / Interview-Ready · 10-14 hrs/week · 2-5 projects per topic · 2-4 documentation sources per topic

Every final project requires: Architecture diagram · ADR · Code review checklist · Docs · Tests · Performance notes

What Changed in This Edition

This is a full curriculum review against staff-level backend practice, not a copyedit. **NEW** marks added topics or tracks, **UPDATED** marks deepened ones, **REORDERED** marks resequenced ones, **DECIDED** marks a topic whose fate was previously ambiguous and is now resolved. Unmarked topics carry over unchanged.

- **NEW — Two continuous tracks added.** A weekly DSA Track now starts in Phase 1 instead of waiting for a later part — one problem set per week, sequenced to match what you’re learning in Java at that moment. A parallel Engineering Practice Track introduces ADRs, code review checklists, pairing habits, and design docs starting in Phase 1, so they’re second nature by the time Phase 3’s API work needs them.
- **DECIDED — GraphQL is permanent, not a bolt-on.** Earlier editions treated GraphQL as an isolated topic. It now ships in the Phase 3 final project as ShopFlow’s permanent product-search and cross-domain query surface, gets a resolver-performance pass in Phase 4 (N+1 in GraphQL, DataLoader batching), and stays live through the Phase 5 capstone. REST remains the primary write/command API; GraphQL is the read/query API — a deliberate CQRS-flavored split, explained where it’s introduced.
- **NEW — Domain-Driven Design and CAP theorem awareness** are now explicit topics (Phase 3 and Phase 4), not left implicit. ShopFlow is modeled as bounded contexts (Catalog, Ordering, Identity, Reviews) instead of one undifferentiated app.
- **NEW — Spring Security depth added:** the filter chain is now taught explicitly before JWT is layered on top, OAuth2/OIDC gets a full topic, and Passkeys/WebAuthn awareness is introduced as the direction the industry is moving beyond passwords.
- **NEW — Testing is now three-layered on purpose:** MockMvc for in-process controller tests, REST Assured for black-box HTTP contract tests, and Testcontainers for real-database integration tests — taught as three different jobs, not three names for the same thing.
- **NEW — SQL progression fixed.** A SQL Foundations & Joins topic now precedes JPA, and a dedicated CTEs & Window Functions topic is added before query optimization, so ranking/running-total reporting queries aren’t skipped.
- **NEW — Performance Optimization & JVM Tuning and Load Testing Preparation** (k6/Gatling, baseline-vs-tuned comparisons, SLOs) close Phase 5, so “production-ready” is measured, not assumed.
- **UPDATED — Every final project (Phases 1-5)** now requires the same six deliverables: an architecture diagram, an ADR, a completed code review checklist, short documentation, a test suite, and a written performance-considerations note. See the requirements strip at the end of each final project.
- **UPDATED — Reordering carried over** from the prior revision is preserved: Concurrency follows Streams in Phase 2; Request Validation & Exception Handling precedes REST API Design in Phase 3.

How This Curriculum Works: Two Continuous Tracks

Alongside the five phases, two tracks run continuously from Week 1 to the end. Each phase page below carries a small box for each, showing that week’s focus. Skipping them is possible; it is also how curricula quietly produce engineers who can build a CRUD app but freeze in a system-design interview or a real design-review meeting.

WEEKLY DSA TRACK. One problem set (4-6 problems) per week, timed to what you’re learning in Java that week so the syntax is never the bottleneck. Sequenced, not random:

- Weeks 1-4 (Phase 1): Arrays & strings — two-pointer, sliding window, prefix sums.
- Weeks 5-9 (Phase 2): Hashing, stacks/queues, recursion & backtracking — directly reusing Collections and Streams just learned.
- Weeks 10-13 (Phase 3): Trees & graphs — BFS/DFS, since the DDD bounded-context map and dependency graphs you’ll draw this phase are graphs.
- Weeks 14-17 (Phase 4): Dynamic programming, heaps/priority queues — the same intuition behind query planning and cache-eviction ordering.
- Weeks 18-20 (Phase 5): System-design-style problems — capacity estimation, back-of-envelope math for the load tests you’re about to run.

ENGINEERING PRACTICE TRACK. The habits that separate “I built an app” from “I can work on a team.” Introduced early, required by name in every final project:

- **Architecture Decision Records (ADRs):** a short, numbered document per significant decision — context, options considered, decision, consequences. Your first ADR is written in Phase 1.
- **Code review checklist:** a living checklist (naming, SRP, test coverage, error handling, security) you apply to your own code before calling anything “done,” versioned as it grows through the phases.
- **Pair programming:** at least one paired session per phase (a study partner, mentor, or structured self-review where you narrate design choices out loud) — practiced explicitly around the concurrency debugging and code-review sessions.
- **Design documents:** a one-page doc before you build anything nontrivial — problem statement, proposed approach, alternatives considered, open questions. Required before Phase 3’s API and Phase 5’s hardening pass.
- **Enterprise coding standards:** consistent formatting (Spotless/Checkstyle), package-by-feature project layout, and commit hygiene, introduced in Phase 3 and enforced from then on.

Topics at a Glance

Phase	Focus	Topics
1	OOP Fundamentals	Encapsulation · Inheritance · Abstraction · Polymorphism · Composition · Exceptions · SOLID & Design Patterns · Records & Sealed Classes
2	Core Java Toolkit	2D/3D Arrays · Collections Framework · Generics · Exception Handling Deep Dive · File I/O · Streams · Concurrency & Multithreading · JDBC · Build Tools · Testing (JUnit/Mockito) · Logging
3	Framework & API	Spring Framework & IoC · Feature-Oriented Structure & Standards ·DDD Awareness · Validation & Exception Handling · REST API Design · OpenAPI/Swagger · GraphQL (permanent) · Spring Security & JWT · OAuth2/OIDC · Passkeys/WebAuthn · Testing (MockMvc, REST Assured, Testcontainers preview)
4	Databases & Performance	SQL Foundations & Joins · JPA Mapping · Lazy/Eager & N+1 · CTEs & Window Functions · JPQL/Native Queries · Indexing & Query Optimization · Transactions & Isolation · MongoDB · Redis Caching Strategies · CAP Theorem & Data Trade-offs · Flyway Migrations
5	Production Readiness	Observability & Actuator · Resilience & Rate Limiting · Docker & Docker Compose · Messaging & Events · Performance Optimization & JVM Tuning · Load Testing Preparation

Next: once ShopFlow’s API is live, tested, containerized, load-tested, and production-hardened, move on to Part B — Frontend Engineering.

Phase 1 of 5 — OOP Fundamentals

Encapsulation · Inheritance · Abstraction · Polymorphism · Composition · Exceptions · SOLID · Design Patterns · Records & Sealed Classes

Active recall through code, not passive reading. Build several suggested projects per pillar in IntelliJ before moving on — the point is writing the pattern from memory, not finishing an app. This phase also opens both continuous tracks: your first DSA sets and your first ADR.

THIS PHASE’S DSA FOCUS. Weeks 1–4: arrays & strings — two-pointer technique, sliding window, prefix sums. Do a set after each OOP pillar below; the goal is fluency with loops and array manipulation before Collections replaces raw arrays in Phase 2.

THIS PHASE’S ENGINEERING PRACTICE. ADR-001 — write your first Architecture Decision Record: “Composition vs. inheritance for CartItem.” Context, two options, decision, consequences — half a page is enough. Also start code review checklist v1: naming, single-responsibility, no raw field exposure, at least one test per public method.

Encapsulation

Hiding internal state behind a controlled public interface, so a class’s data can only change through methods that validate it.

Subtopics: private fields · public getters/setters · validation inside setters · immutability basics

1. Bank Account — Private balance, public deposit/withdraw with validation, reject negatives and overdrafts, test edge cases in main().

2. Student Grade Book — Private grades list exposed only through addGrade() (validate 0–100), getAverage(), getHighest() — never expose the raw list.

3. Immutable Point Class — x/y private final, set only via constructor, no setters, translate() returns a new Point instead of mutating.

Study: Oracle: Classes and Objects · Baeldung: Encapsulation in Java

Inheritance

Letting one class reuse and extend another's fields and behavior, modeling an is-a relationship.

Subtopics: extends · super() calls · method overriding · protected access

1. **Animal Hierarchy** — Animal base with name/speak(); Dog, Cat, Bird override speak()/move(); call super() in each constructor.
2. **Employee Payroll** — Employee.calculatePay() overridden differently by FullTime, PartTime, Contractor.

Study: Oracle: Inheritance · Baeldung: Inheritance in Java

Abstraction

Exposing only essential behavior through an abstract class or interface, hiding implementation detail the caller doesn't need.

Subtopics: abstract classes/methods · interfaces · programming to an interface

1. **Payment Gateway** — Abstract Payment.process(); CreditCard, GCash, PayMaya implementations; Checkout depends only on Payment.
2. **Notification System** — Notifier.send(String) interface; Email/SMS/Push implementations; client code depends only on Notifier.

Study: Oracle: Interfaces and Abstract Classes · Baeldung: Abstract Classes

Polymorphism

Calling the same method name on different object types and getting behavior specific to each.

Subtopics: overriding · dynamic dispatch · upcasting · @Override

1. **Discount Engine** — DiscountStrategy.apply(price); Regular/Senior/StudentDiscount; loop a List and print each discounted price.
2. **Game Character Moves** — Abstract Character.attack(); Warrior, Mage, Archer override it; a game loop calls attack() on any Character.

Study: Oracle: Polymorphism · Baeldung: Polymorphism in Java

Composition

Building complex objects out of smaller ones (has-a) instead of inheritance (is-a) — often the more flexible choice, and the one ADR-001 asks you to defend.

Subtopics: has-a relationships · delegation · composition vs. inheritance

1. **Computer Builder** — Computer has-a CPU, RAM, Storage, GPU; build two configs by swapping components.
2. **Car Engine Swap** — Engine interface with start(); Car composes an Engine field; GasEngine/ElectricEngine swappable at runtime.

Study: GeeksforGeeks: Composition in Java · Refactoring.Guru: Inheritance vs. Composition

Exceptions

Signaling and handling error conditions with a structured, typed alternative to error codes or silent failure — the foundation this curriculum's error-handling story builds on all the way to Phase 3's global handler.

Subtopics: try/catch/finally · custom exceptions · throwing vs. catching · exception hierarchies

1. **ATM Simulator** — InsufficientFundsException, InvalidPINException, DailyLimitExceededException — thrown and caught distinctly.
2. **Resource Cleanup Demo** — Force an exception mid-processing; use try-with-resources to guarantee cleanup.

Study: Oracle: Exceptions · Baeldung: Exception Handling in Java

SOLID Principles & Design Patterns

Apply SOLID across everything you write; implement at least one pattern per category.

Subtopics: SRP · Open/Closed · Liskov · Interface Segregation · Dependency Inversion · Creational/Structural/Behavioral patterns

1. **Factory Method: ProductFactory** — Centralize creation; callers never call `new` directly.
2. **Strategy: DiscountStrategy Swap** — Inject a different strategy into the same Cart at runtime, no if/else chains.
3. **Decorator: Coffee Order Add-ons** — Stack decorators over a base Coffee.cost(); confirm the total adds up.

Study: Refactoring.Guru: Design Patterns · Baeldung: SOLID in Java

Dependency Inversion here is the exact idea Spring's IoC container automates in Phase 3 — note that connection now; it's called out explicitly there.

Records & Sealed Classes (Modern Java) — NEW

Java 14+ records and Java 17+ sealed classes/interfaces — idiomatic for DTOs and closed domain hierarchies, and expected reading in any current Spring Boot codebase.

Subtopics: record basics · compact constructors & validation · sealed interfaces + permits · pattern matching for switch

- 1. DTO Conversion Pass** — Rewrite 3 immutable classes (e.g. Point, an Address) as records with compact-constructor validation; confirm equals/hashCode/toString are free.
- 2. Sealed Payment Result** — Sealed interface PaymentResult permitting Success/Declined/Error records; exhaustive switch expression; confirm the compiler flags a missing case.

Study: Oracle: Record Classes · Oracle: Sealed Classes · Baeldung: Java Records

Final Project — ShopFlow: Foundation (Mini E-Commerce Core)

Console-based shopping cart, pure Java, no Spring Boot yet — forces you to apply everything from this phase.

- Product hierarchy: PhysicalProduct, DigitalProduct (inheritance + polymorphism)
- CartItem composes Product + quantity + discount (composition), represented as a record where it stays a pure value
- DiscountStrategy interface: PercentOff, FixedOff, NoDiscount (Strategy pattern)
- ProductFactory.create(type) to instantiate the correct subclass (Factory Method)
- Private cart total, expose getTotal() only (encapsulation)
- Custom exceptions: OutOfStockException, InvalidQuantityException
- Checkout outcome modeled as a sealed PaymentResult

EVERY FINAL PROJECT SHIPS WITH: Architecture diagram — class/composition diagram of ShopFlow: Foundation · ADR — ADR-001: composition vs. inheritance for CartItem · Code review checklist v1 applied to your own code · README documenting how to run it · JUnit tests for every public method on the core classes · Performance note: Big-O of your cart/discount operations

Phase 2 of 5 — Core Java Toolkit

Collections · Generics · Exceptions · File I/O · Streams · Concurrency · JDBC · Build Tools · Testing · Logging

The layer most self-taught devs skip, then feel shaky about in interviews — and exactly what Spring Boot, JPA, and your own future code lean on constantly. Concurrency is sequenced after Streams so async patterns like CompletableFuture build on functional idioms you’ve just practiced.

THIS PHASE’S DSA FOCUS. Weeks 5–9: hashing, stacks/queues, recursion & backtracking — directly reusing the Collections Framework and Streams you’re learning here. Time your HashMap-heavy DSA sets to land right after the Collections topic below.

THIS PHASE’S ENGINEERING PRACTICE. Pair programming session: debug the intentional race condition in the Concurrency project with a partner (or narrate your reasoning out loud, recorded, and review it after). ADR-002 — “ExecutorService thread pool vs. virtual threads for ShopFlow’s I/O-bound work,” written after the Virtual Threads Benchmark project.

2D & 3D Arrays

Arrays of arrays — the foundation for grids, matrices, and board-shaped data before reaching for Collections.

Subtopics: declaring int[][]/int[][][] · nested-loop traversal · jagged arrays · matrix operations

- 1. Matrix Addition & Multiplication** — addMatrices() and multiplyMatrices() with the row-by-column algorithm.
- 2. Tic-Tac-Toe Board** — char[3][3] board; placeMove(row,col,player); checkWinner() scanning rows/cols/diagonals.

Study: Oracle: Arrays · Baeldung: Multi-Dimensional Arrays

Java Collections Framework

The standard library’s ready-made data structures — List, Set, Map, Queue — that replace most hand-rolled array logic.

Subtopics: List/Set/Map implementations · Queue/Deque · Iterator · Comparable vs. Comparator · time complexity per structure

- 1. Inventory Manager** — Store products in a List, index by ID in a Map, compare lookup approaches conceptually.
- 2. Task Scheduler** — Task with a priority field; PriorityQueue; poll tasks off in priority order.
- 3. Big-O Lookup Table** — Write get/insert/delete/search complexity for ArrayList, LinkedList, HashMap, TreeMap, HashSet, ArrayDeque from memory; verify against the Javadoc; keep as a standing reference sheet.

Study: Oracle: Collections Framework · Baeldung: Guide to ArrayList

Generics

Writing type-safe, reusable classes and methods that work across many types without casting.

Subtopics: generic classes · bounded types · wildcards (? extends / ? super) · generic interfaces

- 1. **Generic Repository Interface** — Repository<T,ID> with save/findById/findAll, implemented in-memory with a Map — previews Spring Data.
- 2. **Generic Stack Implementation** — Stack backed by ArrayList; push/pop/peek/isEmpty; test with Integers and Strings.

Study: Oracle: Generics · Baeldung: Java Generics

Exception Handling — Deep Dive

Beyond try/catch basics: mechanics that matter for production code and interviews.

Subtopics: checked vs. unchecked · try-with-resources · multi-catch · exception chaining

- 1. **Exception Chaining Logger** — Wrap a low-level exception via new CustomEx(msg, cause); log the full chain with getCause().
- 2. **Checked vs. Unchecked Refactor** — Convert a checked exception to a RuntimeException subclass; discuss which fits better here.

Study: Baeldung: Checked vs. Unchecked · Baeldung: try-with-resources

File I/O

Reading and writing data outside the running program — text files, CSVs, and config files.

Subtopics: java.io vs. java.nio.file · BufferedReader/Writer · CSV round-trips · serialization basics

- 1. **CSV Import/Export** — Round-trip your product catalog through a file with BufferedReader/Writer.
- 2. **Directory Walker** — Files.walk() a project directory, filter to .java files, print the 3 largest by size.

Study: Oracle: File I/O · Baeldung: Write to File

Java Streams

A functional-style pipeline for processing collections — source, then a chain of operations, then a result.

Subtopics: map/filter/sorted/distinct/flatMap · Collectors (toList, groupingBy, joining) · reduce · parallelStream() and when not to use it

- 1. **Product Query Pipeline** — filter() by category, map() to a DTO, sorted() by price, collect() to a List.
- 2. **Sales Report Grouping** — groupingBy(category) with downstream summingDouble(price).
- 3. **Parallel vs. Sequential Benchmark** — Time stream() vs. parallelStream() on ~100 items, then ~5,000,000; observe parallelStream only wins at scale; write two bullet points on when to reach for it.

Study: Oracle: java.util.stream · Baeldung: Streams Intro · Baeldung: Parallel Streams

Concurrency & Multithreading — UPDATED

Beyond thread-pool basics: the modern concurrency toolkit a 2026 backend job expects, sequenced after Streams so the functional style is already second nature.

Subtopics: Thread vs. Runnable · ExecutorService & pools · race conditions · synchronized · CompletableFuture · virtual threads (Project Loom) · ConcurrentHashMap & atomics

- 1. **Parallel File Processor** — Fixed thread pool, one task per file, collect with Future, shut down cleanly.
- 2. **Race Condition Demo, Then Fix It** — Share an unsynchronized counter across 100 threads, observe the wrong count, add synchronized, confirm the fix and note the performance trade-off. Pair on this one.
- 3. **CompletableFuture Chain** — supplyAsync() two slow methods, combine with thenCombine(), add exceptionally(); compare readability to nested callbacks.
- 4. **Virtual Threads Benchmark** — 10,000 blocking tasks on a platform-thread pool vs. Executors.newVirtualThreadPerTaskExecutor(); compare thread count and completion time; write ADR-002 from this.

Study: Oracle: Concurrency · Baeldung: ExecutorService · Baeldung: CompletableFuture · JEP 444: Virtual Threads

JDBC

Raw, unabstracted database connectivity — do this before Spring Data/Hibernate so you see exactly what JPA later hides.

Subtopics: DriverManager & Connection · PreparedStatement · ResultSet mapping · commit/rollback

- 1. **Raw JDBC CRUD** — Connect to PostgreSQL with DriverManager; INSERT via PreparedStatement; SELECT and map rows to objects.
- 2. **SQL Injection Demo (Then Fix It)** — Concatenated-string query breaks under an injection payload; rewrite with PreparedStatement placeholders.

Study: Oracle: JDBC Basics · Baeldung: Intro to JDBC · PostgreSQL JDBC Docs

Build Tools

Maven and Gradle — used on every Java job, since Spring Boot itself is generated from a template built on one of them.

Subtopics: Maven (pom.xml) · Gradle (build.gradle) · dependency management · standard project layout

- 1. **Convert to Maven** — Standard src/main/java + src/test/java layout, a pom.xml, one real dependency, build with mvn package.
- 2. **Multi-Module Maven Project** — Parent pom with core and api modules; api depends on core; build from the parent.

Study: Apache Maven: Getting Started · Baeldung: Intro to Maven

Testing (JUnit & Mockito)

Writing automated tests that prove your code works, and keep proving it after every change.

Subtopics: JUnit 5 (@Test, assertions) · Mockito (@Mock, when/thenReturn) · Arrange-Act-Assert · unit vs. integration tests

- 1. **Mock the Repository** — Mock a Repository dependency with @Mock, stub with when().thenReturn(), verify() call count.
- 2. **Parameterized Tests** — @ParameterizedTest with @CsvSource across 5 inputs, both valid and invalid.

Study: JUnit 5 User Guide · Mockito Docs · Baeldung: Mockito Series

Logging

Replacing println debugging with real, leveled, configurable logging — how production code actually reports what’s happening.

Subtopics: log levels · SLF4J + Logback · log patterns · what never to log

- 1. **Add Logging to ShopFlow** — SLF4J + Logback; replace println with logger.info()/debug(); log the exception on every catch block.
- 2. **Redact-Sensitive-Data Filter** — Log a user object, confirm a password field would leak, mask it with a DTO/toString override — this habit is revisited under structured JSON logging in Phase 5.

Study: SLF4J User Manual · Logback Documentation

Final Project — ShopFlow: Core Java Service Layer

Same ShopFlow domain, now backed by real collections, real files, real Streams, real async composition, and a real (if manual) database connection.

- Data layer rebuilt on Collections (Map catalog) and a 2D-array-backed inventory grid
- Generic Repository implemented in-memory, ready to be swapped for JPA
- Full custom exception hierarchy (ShopFlowException base) used consistently
- Catalog filtering/search/reporting rewritten with Streams, including one parallelStream() benchmark note
- Raw JDBC version persisting products to PostgreSQL — no ORM yet
- One checkout-adjacent operation (e.g. confirmation email) implemented with CompletableFuture instead of a blocking call
- Restructured as a Maven build with a src/test/java suite of JUnit + Mockito tests
- All console output replaced with SLF4J logging at appropriate levels

EVERY FINAL PROJECT SHIPS WITH: Architecture diagram — layered service diagram (Repository → Service → CLI) · ADR-002 — thread pool vs. virtual threads · Code review checklist v1, applied and signed off · README with setup + how tests are run · JUnit/Mockito suite covering the service layer · Performance note: parallelStream benchmark + thread-pool sizing rationale

Phase 3 of 5 — Framework & API

Spring & IoC · Feature Structure · DDD · REST · OpenAPI · GraphQL · Security · OAuth2 · Passkeys · Testing

This is where your OOP and Core Java knowledge gets wired into a real framework and a real API surface — and where the Engineering Practice track stops being optional. Validation & global exception handling is built first so every later topic reuses one error shape instead of inventing it twice. Build and test each endpoint in Postman/REST Assured before moving to the next.

THIS PHASE’S DSA FOCUS. Weeks 10-13: trees & graphs — BFS/DFS. Directly useful: the bounded-context map you’ll draw for DDD and any dependency graph between Spring beans are graphs; think of graph traversal as literal practice for reading a Spring context.

THIS PHASE’S ENGINEERING PRACTICE. Design document required before you write the REST API: one page — problem statement, proposed resource model, alternatives considered (REST-only vs. REST+GraphQL), open questions. Enterprise coding standards land here: Spotless/Checkstyle enforced from this point forward, and the project is restructured package-by-feature. Code review checklist grows to v2 (adds: consistent error shape used, no secrets logged, security annotations present).

Spring Framework & IoC Foundations

The container that wires your objects together, so classes declare what they need instead of constructing it themselves. This is the direct successor to the Dependency Inversion and Factory Method patterns you practiced by hand in Phase 1 — Spring’s ApplicationContext is doing at scale what your ProductFactory did manually.

Subtopics: IoC container & ApplicationContext vs. BeanFactory · bean lifecycle & scopes · @Component/@Service/@Repository stereotypes · constructor injection over field injection · @Autowired · what @SpringBootApplication actually assembles (component scan + autoconfiguration) · Controller → Service → Repository layering

1. First Spring Boot App, Traced End to End — Generate via Spring Initializr; add one @RestController; run and hit it; add a @Service and inject it via the constructor; then explicitly diagram which of your Phase 1 manual-wiring code this replaces.

2. Three-Layer Wiring Practice — Repository interface with an in-memory impl → Service depending on it → Controller depending on the Service, all constructor-injected.

Study: Spring Boot Reference Docs · Spring Framework: Core Technologies (DI)

Feature-Oriented Project Structure & Enterprise Standards — NEW

How real teams organize a codebase so it doesn't collapse under its own package structure at 200 endpoints.

Subtopics: package-by-feature vs. package-by-layer · Spotless/Checkstyle setup · consistent commit conventions · code review checklist as a living document

1. Restructure ShopFlow Package-by-Feature — Convert `com.shopflow.{controller,service,repository}` into `com.shopflow.{catalog,ordering,identity}`, each self-contained; explain the trade-off vs. layered packaging in one paragraph.

2. Wire Up Spotless/Checkstyle — Add a Maven plugin enforcing formatting; fail the build on violation; run it in CI (or a pre-commit hook) going forward.

Study: Spotless Maven Plugin Docs · Checkstyle Documentation

Domain-Driven Design Awareness — NEW

Enough DDD to model ShopFlow as bounded contexts instead of one undifferentiated app — this is what your feature packages in the previous topic are actually named after.

Subtopics: bounded contexts & ubiquitous language · entities vs. value objects (tie back to records) · aggregates & aggregate roots · anti-corruption layers (awareness only)

1. Bounded Context Map — Draw ShopFlow as four bounded contexts — Catalog, Ordering, Identity, Reviews — with the calls between them; identify Order as an aggregate root over OrderItems.

2. Entity vs. Value Object Pass — Classify every domain class you've written so far as an entity (has identity/lifecycle) or a value object (defined by its data — a natural fit for the records from Phase 1).

Study: Martin Fowler: Bounded Context · Eric Evans, Domain-Driven Design (reference chapters on aggregates)

Request Validation & Exception Handling — REORDERED

Rejecting bad input before it reaches business logic, and turning every failure into one clean, structured response — built once here, reused by every topic after it.

Subtopics: @Valid & Bean Validation · custom constraint annotations · @ControllerAdvice · @ExceptionHandler

1. Validated Registration Endpoint — @NotBlank/@Email/@Size on a RegisterRequest DTO; @Valid on the controller param; inspect the default validation error.

2. Global Exception Handler — @ControllerAdvice with one ErrorResponse DTO (status, message, timestamp); handlers for validation errors, ShopFlowException, and a catch-all 500 — this becomes the one error shape the whole API returns, including GraphQL error extensions later.

3. Field-Level Error Messages — Loop field errors from MethodArgumentNotValidException into a field→message map in the response body.

Study: Spring: Validation · Baeldung: Validation in Spring Boot · Baeldung: @ControllerAdvice

REST API Design

Designing HTTP endpoints that follow predictable, resource-based conventions — informed by the bounded contexts drawn above, so /products, /orders, and /users map to Catalog, Ordering, and Identity respectively.

Subtopics: HTTP verbs & status codes · resource naming · DTOs vs. entities · pagination & filtering · idempotency for POST/PATCH · API versioning strategy

1. Todo REST API — GET/POST/PUT/DELETE for a Todo resource; correct status codes (200/201/204/404); a DTO separate from the entity.

2. Product Catalog API — GET /products with page/size, sorting, filtering by category; wrapped response with total count + page data.

3. Idempotency Key Support — Idempotency-Key header on POST /orders; store seen keys with their response; repeated key returns the original response instead of a duplicate order; explain why this matters for flaky client networks.

Study: MDN: HTTP Overview · Spring: Building a RESTful Web Service · restfulapi.net

API Documentation with OpenAPI/Swagger

A REST API without generated, browsable documentation is incomplete by industry standard.

Subtopics: springdoc-openapi setup · @Operation/@Schema annotations · Swagger UI · documenting error responses

1. Document the Product Catalog API — Add springdoc-openapi-starter-webmvc-ui; open /swagger-ui.html; confirm every REST API Design endpoint appears.

2. Annotate for Clarity — @Operation summaries, @Schema field descriptions, document the standard ErrorResponse as a reusable response.

Study: springdoc-openapi Docs · OpenAPI Specification · Swagger UI Docs

GraphQL — Permanent Integration — UPDATED

Decision made explicit: GraphQL is not a side exercise. It becomes ShopFlow's permanent read/query API, sitting alongside REST as the write/command API — a deliberate CQRS-flavored split. It's the natural fit for a client that wants a Product plus its nested Reviews (Catalog + Reviews contexts) in one round trip, which a REST endpoint would otherwise need two calls or a bespoke aggregate endpoint for.

Subtopics: schema-first design · queries vs. mutations · resolvers/DataFetchers · REST vs. GraphQL trade-offs · why it stays (cross-context reads) vs. why REST stays (idempotent, cacheable writes with clear HTTP semantics)

1. GraphQL Product Search Schema — .graphqls schema for Product with nested reviews; Query for products(filter) and product(id); resolver reuses your existing Catalog Service.

2. Cross-Context Query — A single GraphQL query returning a Product with its Reviews and average rating, sourced from two different bounded contexts through two resolvers — the exact query REST would need two round trips for.

Study: GraphQL.org: Official Learning Guide · Spring for GraphQL Reference

Spring Security & JWT Authentication — UPDATED

Stateless authentication: the server issues a signed token instead of a session, and every request proves identity by presenting it. Taught security-filter-chain-first, so JWT is understood as one authentication mechanism plugged into that chain, not a magic annotation.

Subtopics: the Spring Security filter chain, explained · SecurityFilterChain bean configuration · token generation & signing · securing endpoints · refresh tokens · CORS configuration · method security (@PreAuthorize)

1. Trace the Filter Chain — Add spring-boot-starter-security with no config, hit an endpoint, observe the default 401, then step through which filters ran — before writing a single line of JWT code.

2. User Auth Service — /register hashing with BCrypt; /login issuing a signed JWT; /me requiring a valid token.

3. Role-Based Endpoint Protection — Role claim in the JWT; @PreAuthorize(ADMIN) on one endpoint; confirm a USER token gets a 403.

4. CORS Configuration — CorsConfigurationSource bean allowing only the Part B frontend's origin; confirm a browser preflight succeeds and an unlisted origin is rejected.

Study: jwt.io: Introduction to JWT · Spring Security Reference · Baeldung: JWT with Spring Security

OAuth2 & OpenID Connect

The delegated-auth pattern most companies actually run in production instead of hand-rolled JWT.

Subtopics: authorization code flow · spring-boot-starter-oauth2-client · ID tokens vs. access tokens · when to choose OAuth2/OIDC over hand-rolled JWT

1. OAuth2/OIDC Login Path — Register a free OAuth2 client (Keycloak locally, or Spring Authorization Server); wire a "Login with Google/Keycloak" flow alongside your JWT login; write 3 bullet points on when a team would pick this over hand-rolled JWT.

Study: Spring Security Reference: OAuth2 · Spring Authorization Server Reference

Passkeys & WebAuthn Awareness — NEW

Where authentication is heading beyond passwords entirely — awareness-level, not a full implementation, but you should be able to explain the ceremony and when it fits.

Subtopics: FIDO2/WebAuthn concept · public-key registration & authentication ceremony · why passkeys resist phishing in a way passwords and even OTPs don't · Spring Security's WebAuthn support (awareness)

1. Trace a Passkey Ceremony — Walk through registration (browser generates a key pair, public key sent to server) and authentication (server challenge, browser signs with the private key) on paper or a sequence diagram; no code required.

2. Passwordless Fit Assessment — Write 3 bullet points on whether ShopFlow's /register and /login flows are good passkey candidates today, and what would need to change.

Study: webauthn.guide · FIDO Alliance: Passkeys · Spring Security: WebAuthn Support

Testing Spring Boot Applications — UPDATED

Three different testing jobs, taught as three different jobs: MockMvc runs the controller in-process without a real server; REST Assured hits a real running instance over HTTP like an external client would; Testcontainers (previewed here, full depth in Phase 4) runs against a real containerized database instead of mocks.

Subtopics: @SpringBootTest · MockMvc for controller tests · REST Assured for black-box contract tests · @DataJpaTest · test profiles · the test pyramid applied to ShopFlow

1. Controller Test with MockMvc — @WebMvcTest for one controller, Service layer mocked, assert JSON on GET, assert 400 on a bad POST body.

2. Security-Aware Controller Test — Hit a secured endpoint with no token, assert 401; generate a valid test token, retry, assert 200.

3. REST Assured Contract Test — Boot the app on a random port; write a REST Assured test that given()/when()/then() a full POST /orders → GET /orders/{id} flow purely over HTTP — no Spring test context shortcuts.

Study: Spring Boot: Testing Reference · Baeldung: Testing in Spring Boot · REST Assured Documentation

Final Project — ShopFlow: Backend API Layer

Extends the Phase 2 core-Java service layer into a full Spring Boot API, organized by feature, with REST for commands and GraphQL for cross-context reads.

- Product CRUD (REST): POST /products, GET /products/{id}, PATCH, DELETE
- Cart endpoints: add item, remove item, apply coupon, get total
- Order placement: POST /orders with state transitions (PENDING → CONFIRMED → SHIPPED), idempotent via Idempotency-Key
- GraphQL query surface: product search with nested reviews, spanning Catalog + Reviews contexts, permanently wired (not a demo endpoint)
- JWT auth: register, login, secure all endpoints except GET /products; CORS configured for the Part B frontend; OAuth2/OIDC login path available alongside it
- Global @ControllerAdvice returning one structured JSON error shape, reused by REST and surfaced as GraphQL error extensions
- Full API documented and browsable at /swagger-ui.html
- /actuator/health exposed and returning UP (full depth in Phase 5)
- Package-by-feature layout; Spotless/Checkstyle passing in the build
- Unit tests (Mockito) + MockMvc controller tests + REST Assured contract tests, ≥80% coverage on business logic

EVERY FINAL PROJECT SHIPS WITH: Architecture diagram — bounded-context map + REST/GraphQL request flow · ADR — “REST for commands, GraphQL for cross-context reads” · Code review checklist v2, applied and signed off · Design doc that preceded the build + README · MockMvc + REST Assured test suites · Performance note: connection pool sizing, Tomcat thread pool defaults vs. tuned

Phase 4 of 5 — Databases & Performance Engineering

SQL Foundations · JPA · N+1 · CTEs & Window Functions · Indexing · Transactions · MongoDB · Redis · CAP Theorem · Flyway

The phase most people rush and regret later. Hibernate behavior — lazy loading, session scope, N+1 — causes invisible production bugs. A SQL foundations pass now precedes JPA, because Phase 2’s raw JDBC assumed SQL fluency this curriculum hadn’t actually taught yet. Redis and MongoDB are added once indexing and query optimization are second nature, and CAP theorem gives you the vocabulary to defend why each data store was chosen.

THIS PHASE’S DSA FOCUS. Weeks 14–17: dynamic programming, heaps/priority queues. The same reasoning — overlapping subproblems, ordered eviction — underlies query planning and the cache-eviction policy you’ll configure for Redis this phase.

THIS PHASE’S ENGINEERING PRACTICE. ADR-003 — “SQL vs. NoSQL per bounded context,” written after the CAP Theorem topic, covering why Orders stay in Postgres, Reviews live in MongoDB, and product lookups get a Redis layer. Code review checklist grows to v3 (adds: no N+1 in new endpoints, every multi-step write wrapped in @Transactional with the correct rollback rule).

SQL Foundations & Joins — NEW

The relational fundamentals Phase 2’s raw JDBC work assumed. Covered explicitly here so JPA’s generated SQL is never a black box.

Subtopics: INNER/LEFT/RIGHT/FULL joins · subqueries vs. joins · normalization (1NF–3NF) · primary/foreign keys & referential integrity

1. Join Practice on the Product Schema — Write INNER, LEFT, and a correlated subquery answering the same question three ways over products/categories/orders; compare readability and, later, EXPLAIN ANALYZE cost.

2. Normalize a Denormalized Table — Given one flat orders_flat table, normalize it to 3NF across orders/order_items/products; explain what anomaly each new table prevents.

Study: PostgreSQL Official Docs: Queries · Use The Index, Luke!: Joins

JPA Entity Mapping & Relationships

Turning Java classes into database tables declaratively, and modeling how those tables relate.

Subtopics: @Entity/@Id/@GeneratedValue · @Column/@Table · @OneToMany/@ManyToOne · @ManyToMany · mappedBy & @JoinColumn

1. One-to-Many: Category → Products — @OneToMany on Category, @ManyToOne on Product, mappedBy set correctly, save and fetch back.

2. Bidirectional Relationship Pitfall — Trigger the classic out-of-sync bug, then fix it with a helper method that sets both sides together.

Study: Spring Data JPA Reference · Hibernate ORM Docs · Baeldung: JPA Entity Mapping

Lazy vs. Eager Loading & the N+1 Problem

The Hibernate behavior that causes the most invisible production bugs — and now also the pattern to watch for in GraphQL resolvers.

Subtopics: FetchType.LAZY vs. EAGER · detecting N+1 via SQL logs · JOIN FETCH · @EntityGraph · N+1 in GraphQL resolvers & DataLoader batching

- 1. Reproduce N+1 On Purpose** — Enable Hibernate SQL logging, fetch 10 orders with lazy items, loop .getItems(), count queries — confirm it's 1+N.
- 2. Fix With JOIN FETCH and @EntityGraph** — Compare a custom JOIN FETCH query against @EntityGraph; note the difference in approach.
- 3. N+1 in Your GraphQL Resolver** — Run the Phase 3 product-with-reviews query for 20 products, count queries fired by the reviews resolver, then fix it with a batched DataLoader instead of per-product fetches.

Study: Baeldung: N+1 in Hibernate · Spring Data JPA: @EntityGraph · graphql-java: DataLoader

CTEs & Window Functions — NEW

The SQL that reporting and ranking queries actually need — routinely skipped in self-taught paths, and directly reused by the sales-report and leaderboard-style features a real e-commerce backend needs.

Subtopics: WITH / common table expressions · recursive CTEs · ROW_NUMBER/RANK/DENSE_RANK · running totals with SUM() OVER · PARTITION BY

- 1. Top Product Per Category** — RANK() OVER (PARTITION BY category ORDER BY sales DESC) to find the best-seller per category in one query.
- 2. Running Revenue Total** — SUM(amount) OVER (ORDER BY order_date) for a running daily-revenue total; compare against the equivalent (and worse) self-join.
- 3. Recursive CTE: Category Tree** — A recursive CTE walking a self-referencing categories table (parent_id) to list a full category tree in one query.

Study: PostgreSQL Docs: WITH Queries · PostgreSQL Docs: Window Functions

JPQL & Native Queries

Querying against your entity model (JPQL) or dropping to raw SQL (native, including the CTEs above) when you need to.

Subtopics: @Query with JPQL · native SQL queries · named parameters · when to choose each

- 1. Custom JPQL Finder Methods** — Multi-condition search with named parameters; compare readability vs. a derived method name.
- 2. Native Query for the Window-Function Report** — Expose the Top-Product-Per-Category CTE as a @Query(nativeQuery = true) method, since JPQL can't express window functions.

Study: Spring Data JPA: Query Methods · Baeldung: JPQL vs. Native Query

PostgreSQL Indexing & Query Optimization

Making queries fast at scale — what an index actually does and how to prove a query is using one.

Subtopics: B-tree vs. GIN indexes · EXPLAIN ANALYZE · composite indexes · when an index doesn't help

- 1. Baseline, Then Add an Index** — Seed 100,000+ rows; EXPLAIN ANALYZE baseline; CREATE INDEX on the filtered column; rerun and compare.
- 2. Composite Index for a Multi-Column Query** — Index (category, price); confirm it's used filtering both; test filtering on price alone and note when the index stops helping.

Study: PostgreSQL Docs: Indexes · Use The Index, Luke!

Transactions & Isolation

Grouping multiple database operations so they either all succeed or all fail together, and understanding what “isolation level” is actually protecting you from.

Subtopics: @Transactional · propagation levels · rollback rules · isolation levels (Read Committed, Repeatable Read, Serializable) · dirty reads / phantom reads, by name

- 1. Transfer Funds End-to-End** — Debit one account, credit another, inside one @Transactional method; force a mid-transfer failure; confirm neither balance changed.
- 2. Propagation: REQUIRES_NEW** — Call a REQUIRES_NEW method from within another @Transactional method; fail the outer transaction after; confirm the inner write survived.

Study: Spring: Transaction Management · Baeldung: Spring @Transactional

MongoDB & Document Databases

A schema-flexible alternative to relational tables — storing nested, denormalized documents instead of normalized rows. This is the Reviews bounded context's home, and ADR-003's first case study.

Subtopics: embedding vs. referencing · MongoRepository · custom @Query · aggregation pipelines

- 1. Embedding vs. Referencing** — Embed a short review list inside a Product document; model a User referenced only by ID; explain when you'd choose each.
- 2. Aggregation Pipeline: Average Rating** — Group reviews by productId, compute an average rating via MongoTemplate — this backs the GraphQL reviews resolver from Phase 3.

Redis Caching Strategies — UPDATED

Most hot-path reads in a real backend hit a cache long before touching Postgres. Ties directly back to “measure before optimizing” from the indexing topic, now at the caching layer.

Subtopics: Spring Cache abstraction (@Cacheable/@CacheEvict) · cache-aside vs. write-through vs. write-behind · TTL and eviction strategy (LRU/LFU) · cache stampede awareness

- 1. Cache the Product Catalog Lookup** — @Cacheable on the hot product-lookup method; docker run redis locally; confirm the second call skips the database.
- 2. Cache Eviction on Update** — @CacheEvict on the update method; confirm the next GET reflects the new value, not a stale one; set a TTL as a backstop.
- 3. Before/After Latency Comparison** — Time 100 repeated requests with caching disabled vs. enabled; record the difference in the same format as the earlier EXPLAIN ANALYZE comparison — this feeds Phase 5’s load test baseline.

Study: Spring: Caching Abstraction · Redis Documentation · Baeldung: Spring Boot Redis Cache

CAP Theorem & Data Consistency Trade-offs — NEW

The vocabulary to defend, not just describe, ShopFlow’s data-store choices — the direct input to ADR-003.

Subtopics: Consistency, Availability, Partition tolerance — what each actually means · CP vs. AP systems in practice · where Postgres, MongoDB (replica sets), and Redis each land · eventual consistency and why the Reviews average-rating cache can tolerate it but an order total cannot

- 1. Classify ShopFlow’s Data Stores** — For Postgres (Orders), MongoDB (Reviews), and Redis (product cache), write one paragraph each on which side of CAP it leans and why that’s an acceptable trade-off for that specific data.
- 2. Write ADR-003** — “SQL vs. NoSQL per bounded context” — formalize the classification above into the ADR format from Phase 1.

Study: Gilbert & Lynch: Brewer’s Conjecture and the CAP Theorem (paper) · MongoDB Docs: Replica Set Consistency

Schema Migrations (Flyway)

Version-controlling your database schema the same way you version-control code, so every environment evolves in lockstep.

Subtopics: versioned migration files · the migrate command · safe vs. unsafe migrations · baseline & repair

- 1. First Flyway Migration** — V1__create_products_table.sql; confirm Flyway applies it automatically on startup.
- 2. Rename Column the Safe Way** — Add the new column → copy data in a follow-up migration → drop the old column in a final migration; explain why this beats a direct rename.

Study: Redgate Flyway Docs · Baeldung: Database Migrations with Flyway

Final Project — ShopFlow: Database & Performance Layer

Connects Phase 3’s API to real PostgreSQL, MongoDB, and Redis — all three reproducible with one docker-compose up, and every data-store choice defended in writing.

- Normalized PostgreSQL schema: users, products, categories, orders, order_items
- JPA entities with correct relationships and fetch strategies; N+1 fixed on the order-history endpoint via @EntityGraph, and in the GraphQL reviews resolver via a DataLoader
- Flyway migrations for every schema change from here forward
- One reporting endpoint backed by a window-function CTE (top product per category or running revenue)
- MongoDB collection: product_reviews with embedded rating + text, referenced userId
- Indexes on products.category_id, orders.user_id, orders.created_at; EXPLAIN ANALYZE the 3 most-used queries
- Multi-step operations (checkout, stock transfer) wrapped in @Transactional with correct rollback rules and a stated isolation level
- @Cacheable product lookups backed by Redis, @CacheEvict on update, measured before/after latency
- Repository/integration tests rewritten with Testcontainers against a real containerized Postgres instead of H2
- A docker-compose.yml bringing up Postgres, MongoDB, and Redis together for local development

EVERY FINAL PROJECT SHIPS WITH: Architecture diagram — data-flow diagram across Postgres/Mongo/Redis with CAP annotations · ADR-003 — SQL vs. NoSQL per bounded context · Code review checklist v3, applied and signed off · README covering docker-compose setup + migration workflow · Testcontainers integration suite · Performance note: EXPLAIN ANALYZE before/after + cache latency comparison

Phase 5 of 5 — Production Readiness & Cloud-Native Basics

Observability · Resilience · Docker Compose · Messaging · Performance & JVM Tuning · Load Testing

Closes the gap between “runs in IntelliJ” and “runs like a real service,” without turning this roadmap into a full microservices course. Deep distributed-systems topics (service discovery, API gateways, distributed tracing, Kafka at production scale) are intentionally left for a later, dedicated track. This phase ends with a measured claim, not an assumed one: a load test report comparing before and after your tuning pass.

THIS PHASE’S DSA FOCUS. Weeks 18–20: system-design-style problems — capacity estimation, back-of-envelope math. This is literal preparation for the Load Testing topic: estimating expected RPS and p99 latency targets before you run k6 is the same skill as estimating throughput in a system-design interview.

THIS PHASE’S ENGINEERING PRACTICE. Design document required before the hardening pass: SLOs (e.g. p99 < 300ms, 99.9% availability), rate-limit budget, rollback plan. ADR-004 — “Synchronous request/response vs. async events for order confirmation,” written after the Messaging topic. Code review checklist reaches v4 (adds: health checks present, secrets from environment not code, resource limits set in Docker).

Observability & Actuator

Every real Spring Boot service exposes its own health and metrics — a small add with an outsized payoff. Logging, introduced in Phase 2, now becomes structured and traceable.

- Subtopics:** Spring Boot Actuator · /health, /metrics, /info · Micrometer basics · structured JSON logging & MDC for request tracing
- 1. Expose Actuator Health & Metrics** — /actuator/health and /actuator/metrics; a custom health indicator checking the database connection.
 - 2. Request Tracing with MDC** — Filter generating a requestId per request, placed in the SLF4J MDC; confirm every log line for that request carries it; switch the log pattern to JSON output.
- Study: Spring Boot Actuator Reference · Micrometer Docs · Logback: MDC*

Resilience & Rate Limiting

Once your API calls anything else, it needs to fail predictably instead of cascading. Kept lightweight: pattern awareness, not a full resilience-framework deep dive.

- Subtopics:** circuit breaker pattern · retry with backoff · rate limiting · Resilience4j basics
- 1. Rate-Limit the Product Search Endpoint** — Resilience4j RateLimiter with a requests-per-second cap; confirm requests over the limit get 429 with Retry-After.
 - 2. Circuit Breaker Around an External Call** — Simulate a shipping-quote service failing ~40% of the time; wrap it with a Resilience4j CircuitBreaker; confirm it opens after repeated failures and fails fast instead of hanging.
- Study: Resilience4j Documentation · Baeldung: Resilience4j Guide*

Containerization with Docker & Docker Compose — UPDATED

How the app actually gets run outside your IDE — should happen before Part B begins, so the backend is deployable on its own before a frontend is layered on top.

- Subtopics:** multi-stage Dockerfile · docker-compose for the full local stack · healthchecks & compose profiles · environment-based configuration, never committing secrets
- 1. Dockerize ShopFlow’s API** — Multi-stage Dockerfile (Maven build stage, slim runtime stage with just the JAR); build and run locally; confirm the API responds on the mapped port.
 - 2. Full Local Stack with docker-compose** — Extend the Phase 4 compose file to include the Spring Boot app; wire env vars for DB/Redis/Mongo hosts instead of localhost; add healthchecks so the app waits for Postgres to be ready; bring the whole stack up with one docker-compose up.
- Study: Docker Docs: Dockerfile Reference · Spring Boot: Container Images · Docker Compose Docs*

Messaging & Events (Awareness Level)

- Just enough event-driven thinking to recognize the pattern in an interview or a real codebase — deliberately not a full Kafka-at-scale treatment.
- Subtopics:** publish/subscribe basics · the outbox pattern · at-least-once delivery & idempotent consumers · when to reach for Kafka/RabbitMQ vs. a simple in-process event
- 1. In-Process Domain Events** — Publish an OrderPlaced event via ApplicationEventPublisher; a listener sends a confirmation (reusing the Phase 2 CompletableFuture email-send); confirm order-placement logic doesn’t need to know the listener exists.
 - 2. Outbox Pattern** — An outbox_events table written in the same @Transactional method that saves an Order; a scheduled poller reads unprocessed rows and “publishes” them (a log line is enough); explain why this avoids the dual-write problem. Write ADR-004 from this decision.
- Study: Microservices.io: Transactional Outbox · Spring: Application Events*

Performance Optimization & JVM Tuning — NEW

Where “production-ready” stops being a feeling and becomes a number. Pulls together every performance thread from earlier phases — query plans, caching, thread pools — into one measured pass.

Subtopics: JFR/async-profiler basics · HikariCP connection pool sizing · GC awareness (throughput vs. pause-time trade-offs) · recap: query optimization, N+1, and caching as the first three levers before touching the JVM

1. Profile a Hot Endpoint — Run a Java Flight Recorder session against the product-search endpoint under load; identify the top time-consuming method; fix it (likely a query or cache miss from earlier phases) and reprofile.

2. Tune the Connection Pool — Load-test with the default HikariCP pool size, then a deliberately undersized and oversized pool; record latency/error-rate differences; settle on a sized-for-load configuration and write down the reasoning.

Study: Oracle: Java Flight Recorder · HikariCP: About Pool Sizing

Load Testing Preparation — NEW

Where the whole curriculum’s performance claims get checked against reality. Closes the loop opened by the DSA track’s capacity-estimation problems this same phase.

Subtopics: k6 or Gatling scripting basics · defining SLOs before you test (p50/p95/p99, error-rate budget) · baseline vs. post-optimization comparison · reading a load-test report

1. Baseline Load Test — Write a k6 script hitting GET /products and POST /orders at a target RPS derived from your capacity-estimation DSA exercise; run it before any tuning and record p50/p95/p99.

2. Tuned Load Test & Report — Re-run the identical script after the caching, indexing, and connection-pool work from this phase and Phase 4; produce a before/after report against your stated SLOs.

Study: k6 Documentation · Gatling Documentation

Final Project — ShopFlow: Production Hardening

The checkpoint where ShopFlow stops being a project that only runs in your IDE and becomes something you could hand to another engineer with a single docker-compose up, a README, and a load-test report.

- Actuator health/metrics exposed, with a custom DB health indicator
- Every request traceable through logs via a request-scoped MDC id, output as structured JSON
- Rate limiting on product search; a circuit breaker around one simulated external call
- Dockerfile for the API, folded into the Phase 4 docker-compose stack alongside Postgres/MongoDB/Redis, with healthchecks
- OrderPlaced modeled as an in-process event, plus an outbox table proving the transactional-consistency pattern
- A JFR profiling pass on the highest-traffic endpoint, with the fix applied and reprofiled
- HikariCP pool sized deliberately, with the sizing rationale documented
- A k6 (or Gatling) load-test report: baseline vs. tuned, measured against stated SLOs

EVERY FINAL PROJECT SHIPS WITH: Architecture diagram — full deployment diagram (containers, network, data stores, external calls) · ADR-004 — sync vs. async for order confirmation · Code review checklist v4, applied and signed off · Design doc (SLOs, rollback plan) + README · Full test suite green in the docker-compose stack · Performance note: k6/Gatling before-vs-after report against SLOs

Backend Engineering — Progress Tracker

Check off each topic once you’ve read its concept sections, built its hands-on projects, folded the result into ShopFlow, and — where marked — completed the matching Engineering Practice deliverable.

Phase 1 — OOP Fundamentals

- ☐ Encapsulation
- ☐ Inheritance
- ☐ Abstraction
- ☐ Polymorphism
- ☐ Composition — ADR-001
- ☐ Exceptions
- ☐ SOLID Principles & Design Patterns — Checklist v1
- ☐ Records & Sealed Classes (NEW)

Phase 2 — Core Java Toolkit

- ☐ 2D & 3D Arrays
- ☐ Java Collections Framework
- ☐ Generics
- ☐ Exception Handling — Deep Dive
- ☐ File I/O
- ☐ Java Streams
- ☐ Concurrency & Multithreading (UPDATED) — Pairing session, ADR-002
- ☐ JDBC
- ☐ Build Tools
- ☐ Testing (JUnit & Mockito)
- ☐ Logging

Phase 3 — Framework & API

- ☐ Spring Framework & IoC Foundations
- ☐ Feature-Oriented Structure & Standards (NEW) — *Enterprise standards enforced*
- ☐ Domain-Driven Design Awareness (NEW) — *Bounded context map*
- ☐ Request Validation & Exception Handling (REORDERED)
- ☐ REST API Design — *Design doc*
- ☐ API Documentation with OpenAPI/Swagger
- ☐ GraphQL — Permanent Integration (UPDATED)
- ☐ Spring Security & JWT Authentication (UPDATED)
- ☐ OAuth2 & OpenID Connect
- ☐ Passkeys & WebAuthn Awareness (NEW)
- ☐ Testing Spring Boot Applications (UPDATED) — *Checklist v2*

Phase 4 — Databases & Performance Engineering

- ☐ SQL Foundations & Joins (NEW)
- ☐ JPA Entity Mapping & Relationships
- ☐ Lazy vs. Eager Loading & the N+1 Problem
- ☐ CTEs & Window Functions (NEW)
- ☐ JPQL & Native Queries
- ☐ PostgreSQL Indexing & Query Optimization
- ☐ Transactions & Isolation — *Checklist v3*
- ☐ MongoDB & Document Databases
- ☐ Redis Caching Strategies (UPDATED)
- ☐ CAP Theorem & Data Consistency Trade-offs (NEW) — *ADR-003*
- ☐ Schema Migrations (Flyway)

Phase 5 — Production Readiness & Cloud-Native Basics

- ☐ Observability & Actuator
- ☐ Resilience & Rate Limiting
- ☐ Containerization with Docker & Docker Compose (UPDATED)
- ☐ Messaging & Events — *ADR-004*
- ☐ Performance Optimization & JVM Tuning (NEW)
- ☐ Load Testing Preparation (NEW) — *Checklist v4, Design doc*

Continuous tracks (not individually boxed above): Weekly DSA Track, Weeks 1-20 · Engineering Practice Track — ADR-001 through ADR-004, code review checklist v1-v4, one design doc in Phase 3 and one in Phase 5, at least one paired session per phase.